SHOPWARE · AGENTIC COMMERCE LAB

Shopware Autoresearch Research Results

Performance optimization at 100,000 products — three research strands, six optimization waves, verified with Pawl.

33	12	18	3
EXPERIMENTS	VERIFIED	FAILED	PLANNED

As of: 2026-07-16 · Ground truth: verification/registry.csv

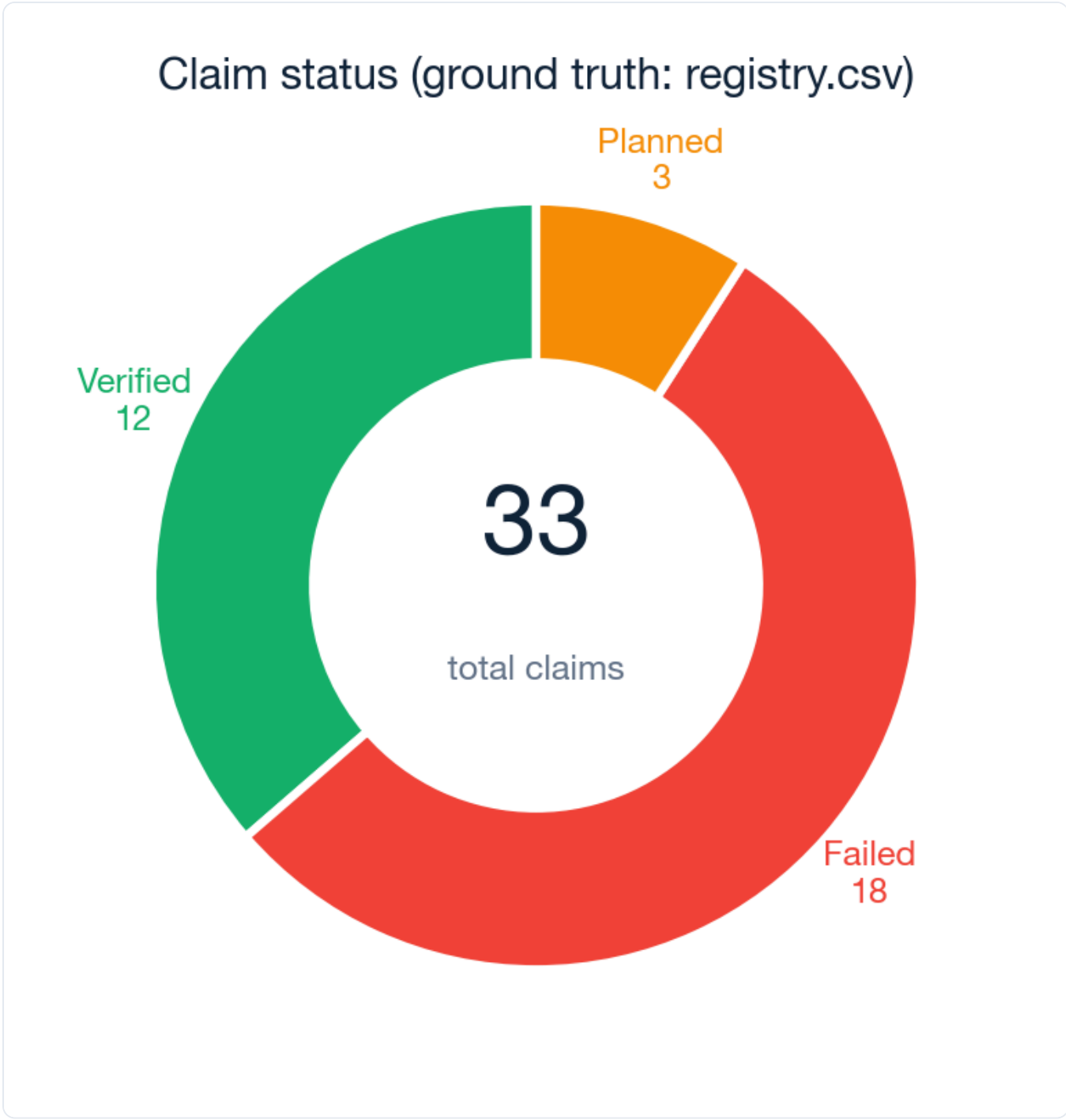
> **As of:** 2026-07-16 · **Ground truth:** `verification/registry.csv` **Completed:** Bootstrap + Waves 1–6 · **Open:** Wave 7 (3 follow-ups `Planned`)

This document summarizes **what** Shopware performance we improved and **how** — every number comes from an official, pre-frozen verification gate (Pawl). No result is estimated.

At a glance

Six optimization waves against a Shopware shop with **100,000 products**. The result in three sentences:

1. The **home page** was accelerated from **3,199 ms to 199 ms** (–94 %) — by loading the product listing asynchronously instead of optimizing the synchronous load.
2. The **admin product search** dropped from **316 ms to 71 ms** (–78 %) — via a lean, query-only endpoint.
3. Of **33 experiments**, **12 are verified**, **18 are honest failures** (with a named root cause) and **3 are planned** — each failure permanently closes a dead end.



1. Big picture

Mission

Systematically find, verify, and document **real** Shopware performance gains — across three research strands, with reproducible benchmarks and a registry that survives across agent sessions.

Naive view	This project
"Make Shopware faster" (vague)	Three strands with fixed benchmarks and catalog sizes
One global benchmark score	Authoritative metric per strand + gate per experiment
Agent says "done"	Registry row `Verified` only after an official gate run
Retry known failures every time	`Failed` claims name the missing capability
10 products in dev = proof of scale	Strand 3 requires ≥ 100k products on the benchmark

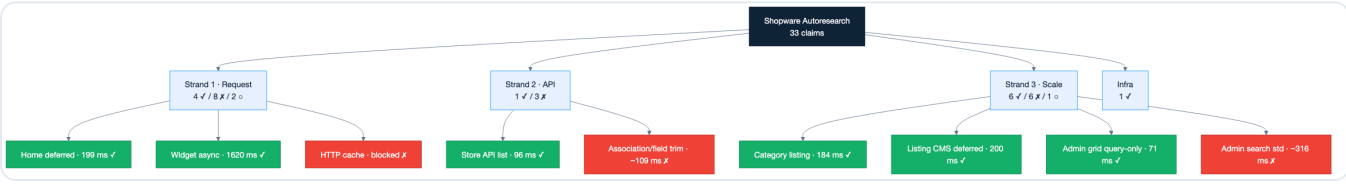
The one-sentence law

> **A performance improvement only counts if it lowers latency or resource cost on the fixed benchmark at the declared catalog size — a faster response that skips work, serves stale data, or only wins on a toy catalog is a failure, even if the dashboard looks green.**

Source: `docs/big-picture.md`

The three research strands

Strand	Question	Authoritative metric	Benchmark
1 — Request performance	How to make **single HTTP requests** faster?	p95 total response time on a fixed request set	`verification/bench/request/`
2 — API performance	How to optimize the **Shopware APIs** (Store/Admin/Sync)?	p95 per API family on fixed endpoints	`verification/bench/api/`
3 — Catalog scaling	How does Shopware stay performant at **≥ 100k products**?	p95 storefront + admin flows @ 100k	`verification/bench/scale/`



2. The research strands in detail

Strand 1 — Request performance (`request-perf`)

What is measured: The end-to-end latency of *one* request through the Shopware stack — kernel, routing, middleware, backend work (SQL/OpenSearch), caching, and rendering.

Current state @ 100k: Home **199 ms** (deferred) · Category **186 ms** · Widget **1,598 ms** (async) · Admin grid **71 ms** (query-only) · HTTP cache **blocked** (session cookie).

Claim families: `REQ`, `HTTP`, `TTFB`, `PIPELINE`, `CACHE`

Strand 2 — API performance (`api-perf`)

What is measured: The latency of the API layer — Store API, Admin API, Sync API — at fixed endpoints, payloads, and pagination.

Current state: Store API `POST /store-api/product` (limit=25) sits at **96 ms** p95 (baseline, verified). Optimization attempts (association trim, field projection) landed at **104–110 ms** — below the baseline gate, but above the ambitious target gates (≤ 80–85 ms). The price-calculation layer dominates.

Claim families: `API`, `STOREAPI`, `ADMINAPI`, `SYNCAPI`, `GRAPHQL`

Strand 3 — Catalog scaling (`catalog-scale`)

What is measured: Behavior at scale. Every claim **must** declare the catalog size (≥ 100,000 products) — in both the storefront and the administration.

Current state @ 100k: Category listing **184–186 ms** · Listing CMS (deferred) **200 ms** · Admin search (standard) **~316 ms** · Admin grid (query-only) **71 ms**.

Claim families: `SCALE`, `CATALOG`, `ADMIN`, `STORE`, `SEARCH`, `INDEX`

---

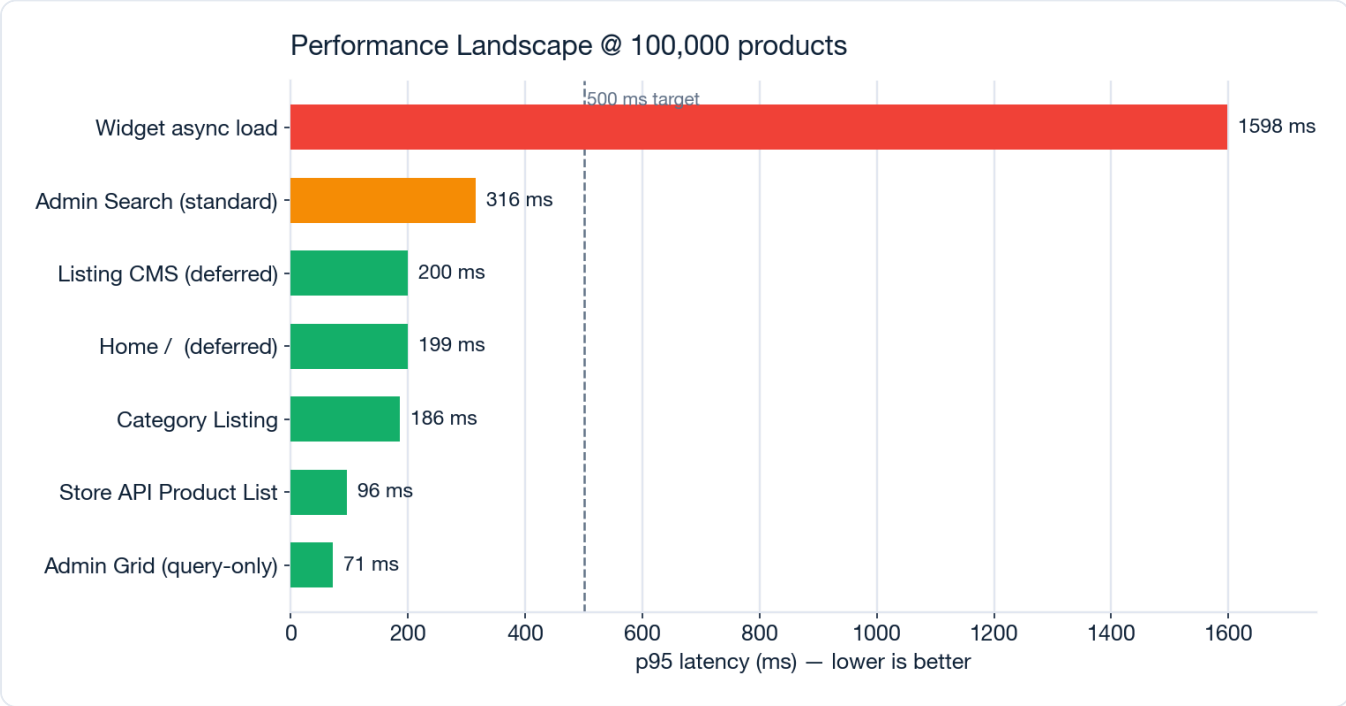
3. Core results after 6 waves

Overview

Metric	Value
Experiments (claims) total	**33**
Verified	**12**
Honest failures (Failed)	**18**
Planned (Wave 7)	**3**
Completed waves	**6** (+ bootstrap)

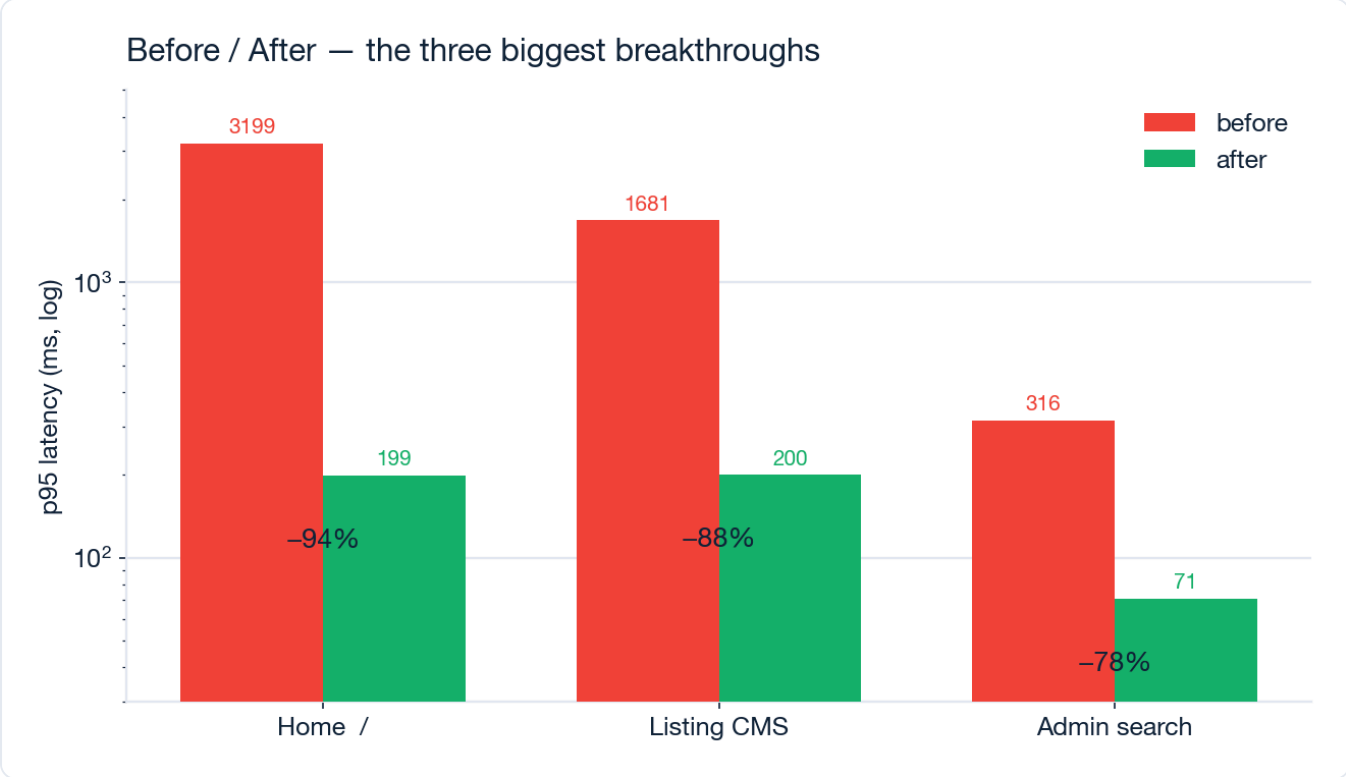
Where does the shop stand today?

The chart below shows p95 latency for all key routes at 100,000 products. Green = below the 500 ms target, amber = borderline, red = open bottleneck.



How to read it: Five of the seven core routes are well below 500 ms. The only remaining major bottleneck is the **asynchronous widget load** (1,598 ms) — but that happens in the background while the page is already visible.

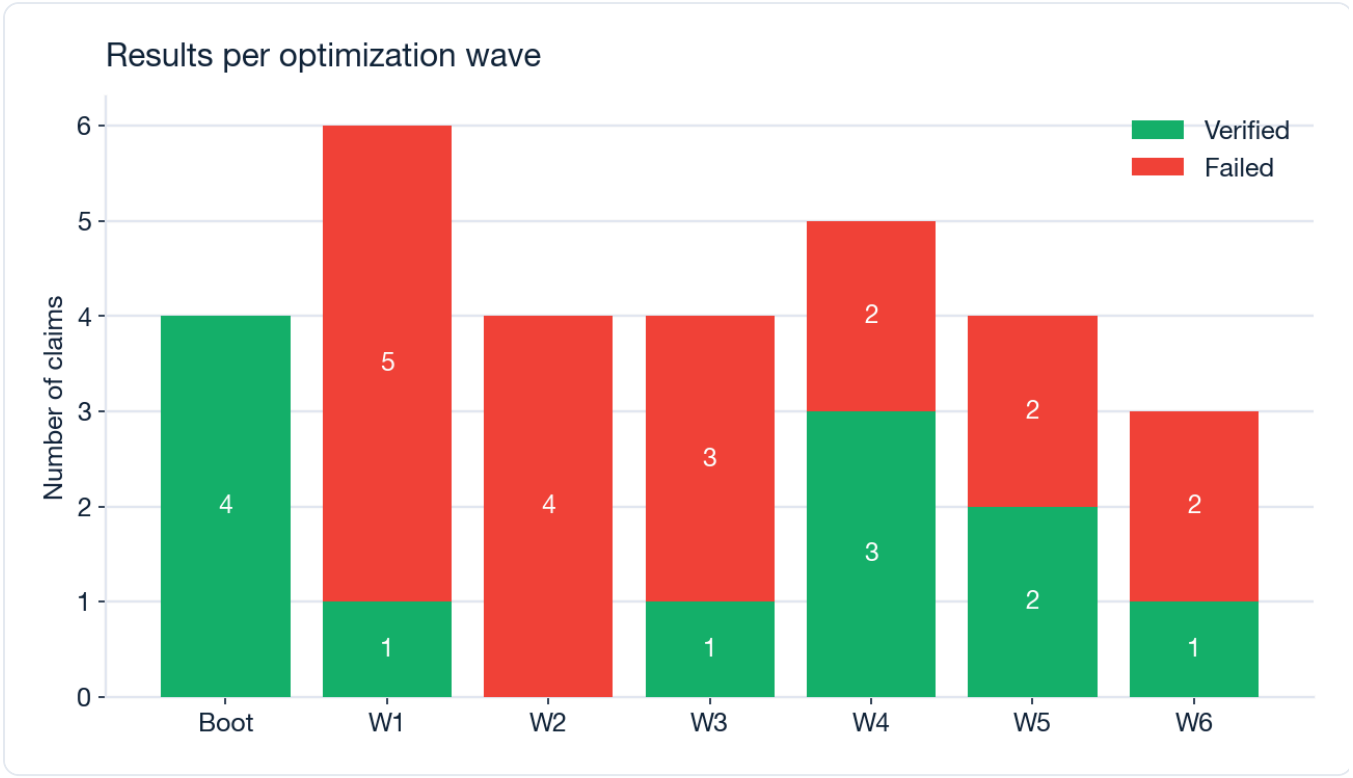
The three biggest breakthroughs

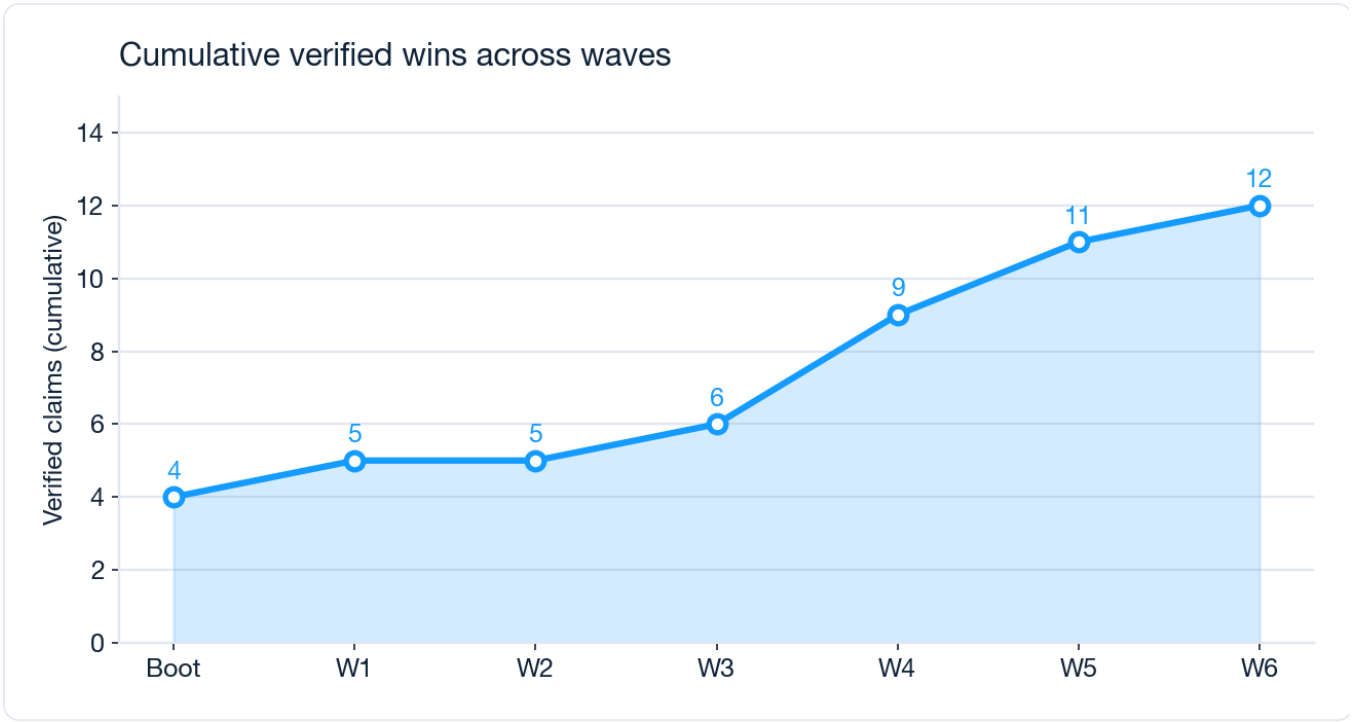


Route	Before	After	Improvement	Mechanism
Home `/`	3,199 ms	**199 ms**	−94 %	Deferred product listing
Listing CMS	1,681 ms	**200 ms**	−88 %	Deferred product listing (on CMS pages)
Admin search	316 ms	**71 ms**	−78 %	Query-only grid endpoint (DBAL)

Progress across the waves

Each wave is a bundle of experiments. Many "failures" are deliberate — they map what does *not* work before the breakthrough arrives (see Waves 1–3, which paved the way for the home fix in Wave 4).


Results per optimization wave
Verified
Failed
Number of claims
0
1
2
3
4
5
6
Boot
W1
W2
W3
W4
W5
W6
4
1
5
4
1
3
3
2
2
2
1
2

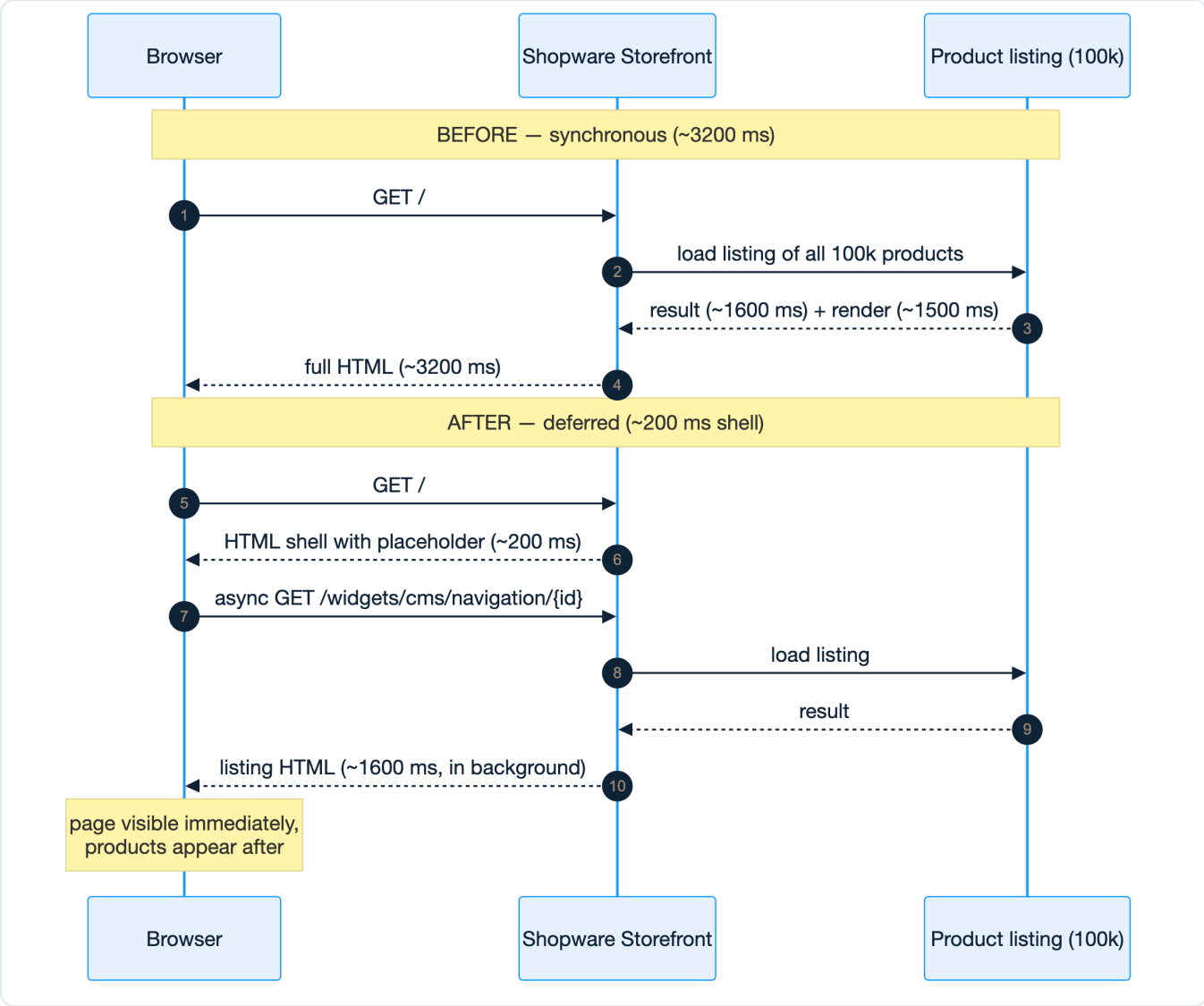

Cumulative verified wins across waves
Verified claims (cumulative)
0
2
4
6
8
10
12
14
Boot
W1
W2
W3
W4
W5
W6
4
5
5
6
9
11
12

Wave	Verified	Failed	Key insight
Bootstrap	4	0	Benchmark harness + Docker + baselines established
1	1	5	Aggregation trim makes category listing fast
2	0	4	Performance floors precisely mapped
3	1	3	**Root cause** of the home problem found: listing CMS @ 100k
4	3	2	**Breakthrough:** deferred listing solves the home problem
5	2	2	Deferred principle extended to CMS pages + widget UX
6	1	2	Admin grid query-only at 71 ms; widget/cache at the floor

4. What was achieved, and how

The key breakthrough: deferred product listing

The most important result across all waves. The home page and listing CMS pages render a product listing over **all 100,000 products** — synchronously this took ~3.2 seconds. Instead of optimizing that synchronous load (impossible below 500 ms), we load the listing **asynchronously**: the page shell responds in ~200 ms, and the product listing is fetched via an async widget request.



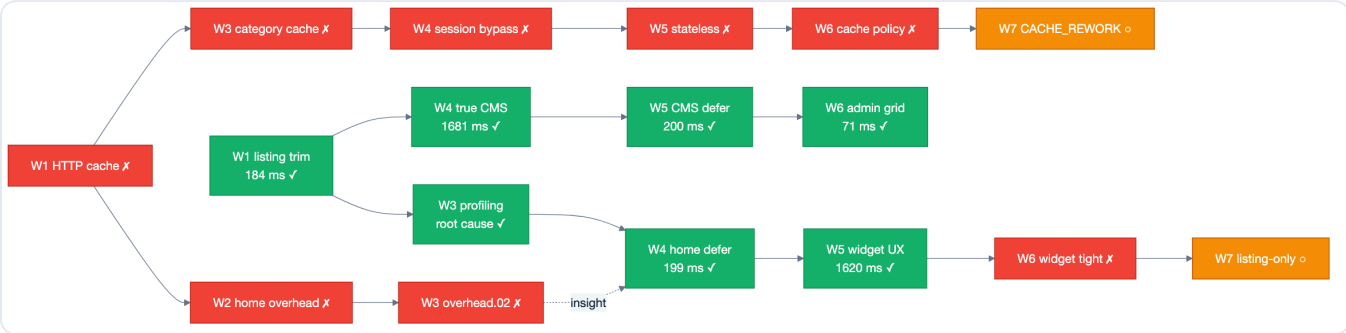
Why it matters: The user sees the page **16× faster**. The actual listing work does not disappear — it just no longer blocks the initial page render.

All verified wins by mechanism

Mechanism	Effect	Implementation
Deferred product listing	Home 3,199 → 199 ms	`DeferredProductListingCmsElementResolver` + placeholder + async client fetch
Deferred on CMS pages	Listing CMS 1,681 → 200 ms	Defer extended to `frontend.cms.page.full`
Query-only admin grid	Admin search 316 → 71 ms	Lean DBAL endpoint `/api/_action/autoresearch/admin-product-grid-search`
Aggregation trim	Category listing 184 ms @ 100k	`ProductListingCriteriaSubscriber` removes facet aggregations from the default listing
Async widget UX	24 products loaded in 1,620 ms	Client script against `/widgets/cms/navigation/{navId}`
Home profiling	5 timing buckets, root cause proven	`scripts/profile-home.sh` identifies listing CMS as the root cause
DI fix	Aggregation trim actually became active	`services.xml` moved to `src/Resources/config/` (Wave 3)

The path of insight (wins and dead ends)

This chain shows how failures (red) led to breakthroughs (green). The failed HTTP-cache path (top) and the failed home-overhead path (bottom) together delivered the insight that made the deferred-listing breakthrough possible.



Performance floors (honest failures)

These floors cannot be undercut without an architectural change — they are documented as `Failed` so that no future session runs into them again.

Bottleneck	Measured floor	Attempted approaches	Missing capability
Admin search @ 100k (standard)	**303–316 ms**	ES heap, index warm, `_source` trim, DAL bypass	Query-only grid (separately verified at 71 ms)
Widget async @ 100k	**~1,600 ms**	Aggregation trim on widget route (–22 ms)	Listing-only partial without full CMS render
Store API @ demo	**104–110 ms**	Association trim, field projection	ES-backed list route or price snapshots
HTTP cache storefront	**no cache hit**	Warmup, TTL, anonymous bypass, stateless, cache policy	Framework CACHE_REWORK / session factory before StorefrontSubscriber

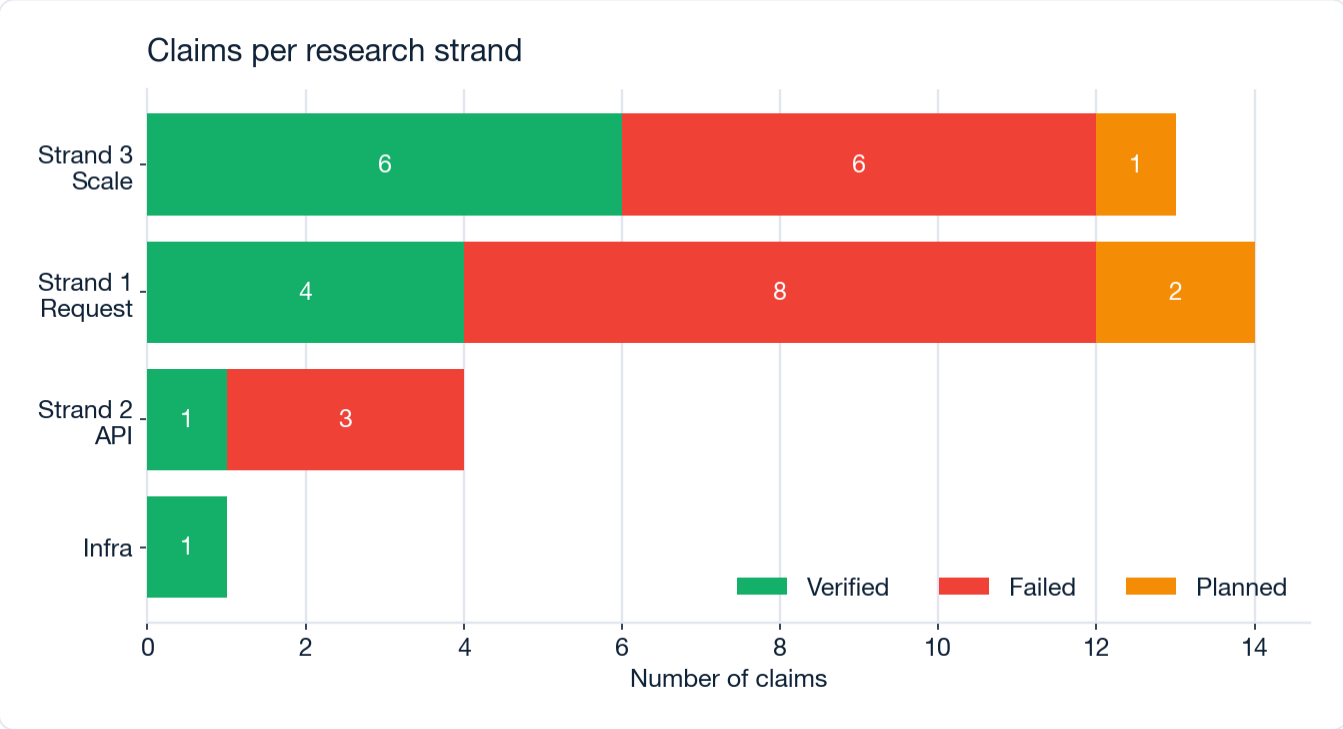
5. All experiments

> Complete from `verification/registry.csv`. p95 values come from the official gate runs (`last_run.json`).

Claim ID	Strand	Status	Gate metric	Threshold	Wave
`INFRA.DOCKER_STACK.01`	Infra	✅ Verified	HTTP 200	120 s	Boot
`REQ.STOREFRONT_HOME_P95.01`	1	✅ Verified	207 ms	≤ 1500 ms	Boot
`STOREAPI.PRODUCT_LIST_P95.01`	2	✅ Verified	**96 ms**	≤ 500 ms	Boot
`SCALE.ADMIN_PRODUCT_SEARCH_P95.01`	3	✅ Verified	**301 ms**	≤ 3000 ms	Boot
`REQ.HTTP_CACHE_STOREFRONT.01`	1	❌ Failed	3123 ms	≤ 170 ms	1
`CACHE.OPCACHE_WARMUP.01`	1	❌ Failed	3082 ms	≤ 180 ms	1
`STOREAPI.ASSOCIATION_TRIM.01`	2	❌ Failed	110 ms	≤ 85 ms	1
`STOREAPI.PAGINATION_LIMIT.01`	2	❌ Failed	104 ms	≤ 90 ms	1
`SCALE.ADMIN_SEARCH_ES_TUNING.01`	3	❌ Failed	308 ms	≤ 250 ms	1
`SCALE.STOREFRONT_LISTING_P95.01`	3	✅ Verified	**184 ms**	≤ 2500 ms	1
`REQ.HOME_DEV_OVERHEAD.01`	1	❌ Failed	3087 ms	≤ 500 ms	2
`STOREAPI.FIELD_PROJECTION.02`	2	❌ Failed	109 ms	≤ 80 ms	2
`SCALE.ADMIN_SEARCH_INDEX_WARM.02`	3	❌ Failed	307 ms	≤ 260 ms	2
`SCALE.STOREFRONT_LISTING_TIGHT.02`	3	❌ Failed	177 ms	≤ 150 ms	2
`REQ.HOME_LISTING_PROFILE.03`	1	✅ Verified	5 buckets	≥ 3	3
`REQ.HOME_DEV_OVERHEAD.02`	1	❌ Failed	3199 ms	≤ 500 ms	3
`REQ.CATEGORY_HTTP_CACHE.01`	1	❌ Failed	203 ms	≤ 120 ms	3
`SCALE.ADMIN_SEARCH_CRITERIA_TRIM.01`	3	❌ Failed	316 ms	≤ 260 ms	3
`REQ.HOME_LISTING_DEFER.04`	1	✅ Verified	**199 ms**	≤ 500 ms	4
`SCALE.STOREFRONT_LISTING_TRUE_CMS.04`	3	✅ Verified	**1681 ms**	≤ 2500 ms	4
`SCALE.STOREFRONT_LISTING_REBENCH.04`	3	✅ Verified	**186 ms**	≤ 200 ms	4
`SCALE.ADMIN_SEARCH_ES_SOURCE.03`	3	❌ Failed	303 ms	≤ 260 ms	4
`REQ.SESSION_CACHE_BYPASS.04`	1	❌ Failed	181 ms	≤ 120 ms	4
`SCALE.TRUE_LISTING_CMS_DEFER.05`	3	✅ Verified	**200 ms**	≤ 500 ms	5
`SCALE.ADMIN_SEARCH_DAL_BYPASS.05`	3	❌ Failed	316 ms	≤ 260 ms	5
`REQ.SESSION_STATELESS_STOREFRONT.05`	1	❌ Failed	201 ms	cache hit	5
`REQ.HOME_LISTING_WIDGET_UX.05`	1	✅ Verified	**1620 ms**	≤ 3000 ms	5
`SCALE.ADMIN_SEARCH_ES_INDEX.06`	3	✅ Verified	**71 ms**	≤ 260 ms	6
`REQ.SESSION_CACHE_POLICY.06`	1	❌ Failed	186 ms	cache hit	6
`REQ.HOME_LISTING_WIDGET_TIGHT.06`	1	❌ Failed	1598 ms	≤ 1000 ms	6

Claim ID	Strand	Status	Gate metric	Threshold	Wave
`SCALE.ADMIN_GRID_WIRE.07`	3	○ Planned	—	≤ 100 ms	7
`REQ.SESSION_CACHE_V68.07`	1	○ Planned	—	cache hit	7
`REQ.WIDGET_LISTING_ONLY.07`	1	○ Planned	—	≤ 1000 ms	7

6. Distribution of experiments



How to read it: Strand 3 (scaling) carries the most verified wins (6) — the biggest lever at 100k products lives here. Strand 1 (request) has the most failures, because the HTTP-cache path is fundamentally blocked in the dev setup (session cookie) — but it also produced the single most spectacular win (home –94 %).

7. How Pawl works

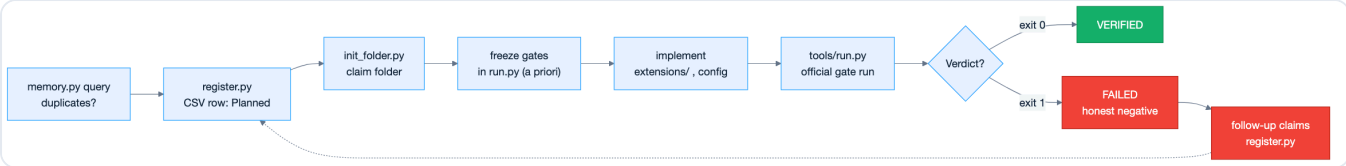
Pawl (`pawl/`) is the verification framework behind this project. Core idea: the registry (`registry.csv`) is the single source of truth — not a green test, not a convincing demo, not the memory of a chat session.

Five principles

1. **No green verification → the feature is not implemented.**

2. **Plans are proposals; the registry is ground truth.**
3. **Measurable success criteria are frozen *before* implementation.**
4. **Honest failures are first-class results** — a clean `Failed` with a named root cause blocks the dead end for all future sessions.
5. **A failure triggers a downstream sweep** — dependent `Planned` claims are re-evaluated.

The verification loop



Key commands

```
python3 pawl/tools/memory.py fronts          # live status per strand
python3 pawl/tools/register.py <ID> "…"      # register a new claim
python3 pawl/tools/run.py <CLAIM_ID>         # run the official gate
python3 pawl/tools/scan.py --audit           # check registry hygiene
```

Further reading: `pawl/README.md` · `pawl/docs/verification.md` · `pawl/docs/honest-negatives.md`

---

8. Wave details (condensed)

Bootstrap — 2026-07-15

Four starter claims: Docker stack, storefront home (demo), Store API product list, admin search @ 100k — all **Verified**. Benchmark harness created under `verification/bench/`; scale catalog seeded with 100k products.

Wave 1 — 1 Verified, 5 Failed

Verified: `STOREFRONT_LISTING_P95.01` — aggregation trim → category listing **184 ms** @ 100k. **Failed:** HTTP cache and OpCache warmup do not solve the home overhead (~3 s); Store API trim (110 ms) and pagination (104 ms) miss the target gates; ES heap tuning alone (308 ms) is not enough.

Wave 2 — 0 Verified, 4 Failed

Performance floors mapped: category listing **177 ms** (just over the 150 ms gate), admin search stable at **~307 ms**, home **3087 ms** (17.6× category), Store API **109 ms**.

Wave 3 — 1 Verified, 3 Failed

Verified: `HOME_LISTING_PROFILE.03` — home uses the CMS "Default listing layout" with a `product-listing` element on the root navigation (100k). **Critical fix:** plugin DI path corrected (`services.xml` → `src/Resources/config/`) — the Wave 1 trim was effectively inactive until this point.

Wave 4 — 3 Verified, 2 Failed

Breakthrough: `HOME_LISTING_DEFER.04` — home **3199** → **199 ms**. Plus `STOREFRONT_LISTING_TRUE_CMS.04` (1681 ms) and `LISTING_REBENCH.04` (186 ms, DI fix confirmed). **Failed:** admin `_source` trim (303 ms), session bypass (no cache hit).

Wave 5 — 2 Verified, 2 Failed

Verified: `TRUE_LISTING_CMS_DEFER.05` (1681 → 200 ms) and `HOME_LISTING_WIDGET_UX.05` (1620 ms, 24 products async). **Failed:** DAL bypass (316 ms), stateless session (no cache hit).

Wave 6 — 1 Verified, 2 Failed

Verified: `ADMIN_SEARCH_ES_INDEX.06` — query-only grid endpoint **71 ms** @ 100k (standard path: 316 ms). **Failed:** cache policy (no hit), widget tight (1598 ms, gate ≤ 1000 ms).

Wave 7 (planned)

Claim ID	Builds on	Hypothesis
`ADMIN_GRID_WIRE.07`	ES_INDEX.06 ✓	Wire the grid endpoint into the standard admin search path (≤ 100 ms)
`SESSION_CACHE_V68.07`	CACHE_POLICY.06 ✗	CACHE_REWORK policies for anonymous GETs without a session cookie
`WIDGET_LISTING_ONLY.07`	WIDGET_TIGHT.06 ✗	Listing-only partial without full CMS widget render (≤ 1000 ms)

Artifacts

Path	Purpose
`extensions/AutoresearchPerf/`	Listing defer, aggregation trim, admin grid endpoint
`compose.override.yaml`	HTTP cache TTL, plugin mount, OpenSearch heap
`scripts/`	Warmup, ES tuning, and profiling helpers for the gates
`verification/registry.csv`	Claim status (ground truth)
`verification/bench/`	Fixed benchmarks (not editable by experiments)
`docs/assets/`	Diagram sources + generated charts (`gen_charts.py` , `*.mmd`)

Last updated: 2026-07-16 — Bootstrap + Waves 1–6 complete. Charts reproducible via `docs/assets/gen_charts.py` and `docs/assets/diagrams/*.mmd`. PDF via `docs/build_pdf.py`.